SD242 – Verificação de Sistemas Digitais (80h)

Autores	
Anderson Max Almeida e Porfirio	

Histórico de revisões		
05/11/2025	V1.0	Versão inicial

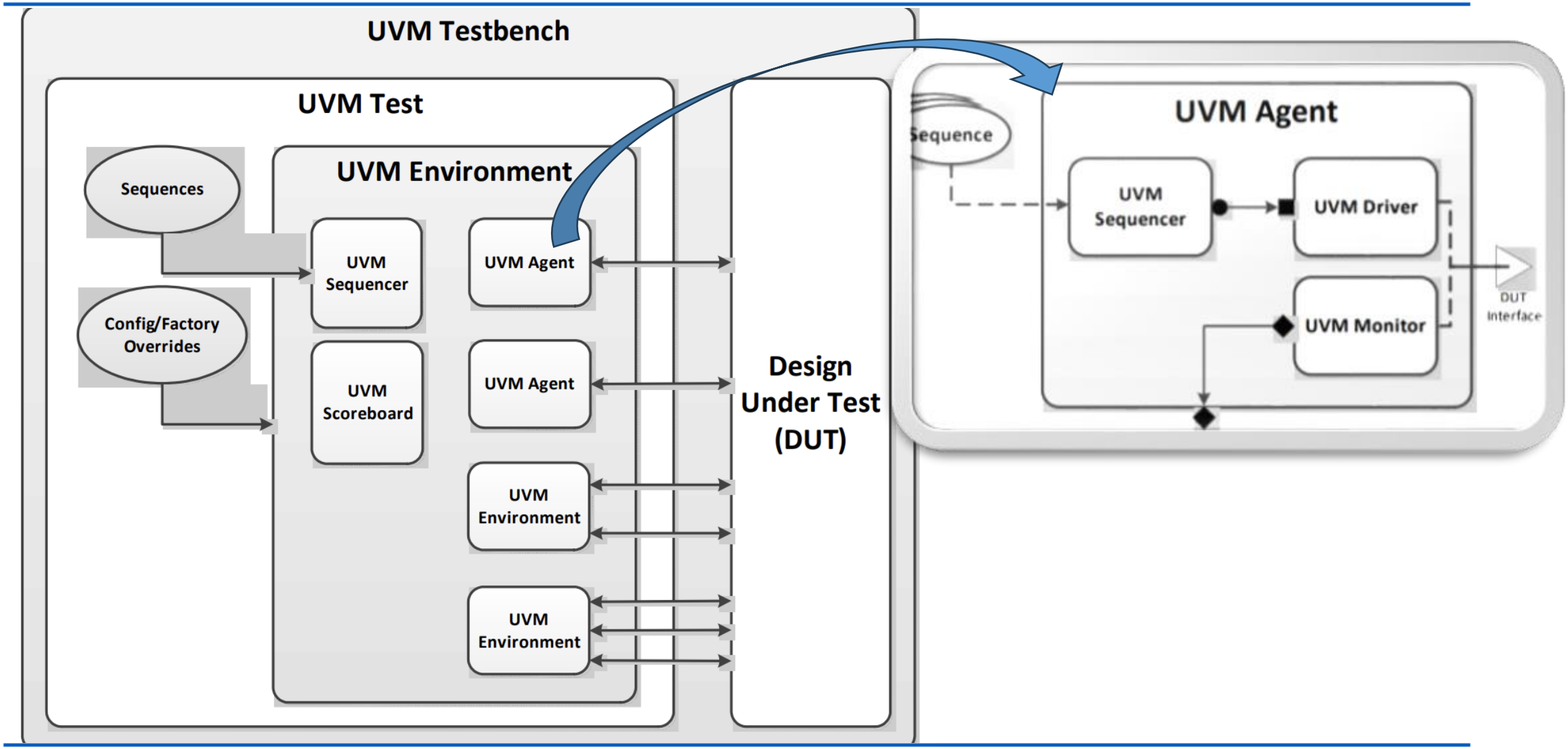
Página em branco.

Aula 18
Sequências UVM

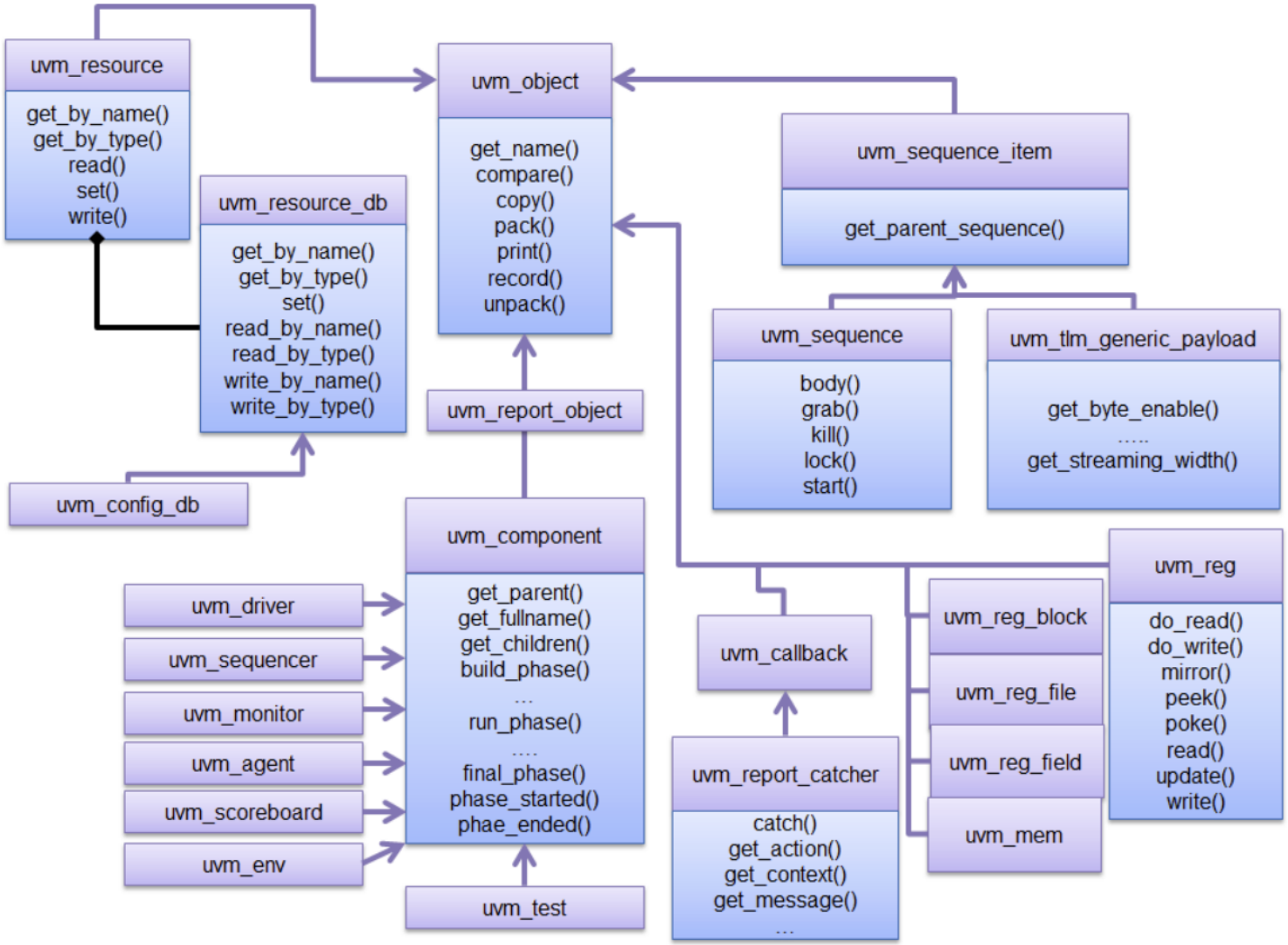
Tópicos

- Testbench UVM
- UVM Components
- Standard UVM Phases
- Sequence UVM
- UVM User guide
- Sequence UVM
- Sequence classes
- Mecanismo de Arbitragem
- Roteamento de Respostas
- Acesso Exclusivo

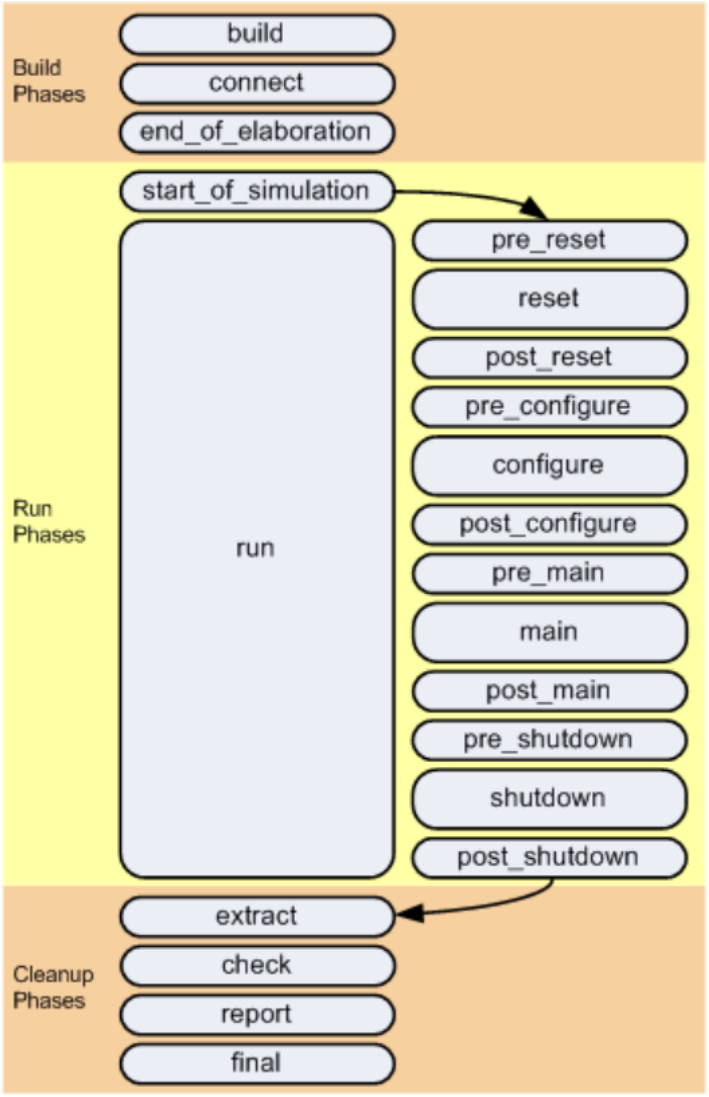
UVM


UVM Testbench
UVM Test
Sequences
Config/Factory Overrides
UVM Environment
UVM Sequencer
UVM Scoreboard
UVM Agent
UVM Agent
UVM Environment
UVM Environment
Design Under Test (DUT)
Sequence
UVM Agent
UVM Sequencer
UVM Driver
UVM Monitor
DUT Interface

UVM Components



Standard UVM Phases



Sequences UVM

- Código exemplo ULA
- Guia UVM.docx (UVM em português - em desenvolvimento)

Sequences UVM

- Sequences classes
  - Sequences encapsulam procedimentos definidos pelo usuário que geram múltiplas transações baseadas no **uvm_sequence_item.**
  - Essas sequences podem ser reutilizadas, estendidas, randomizadas e combinadas de forma sequencial e hierárquica para produzir estímulos para um DUT.
  - A arquitetura de sequência de um driver, sequencer, sequence_item e sequence é orientada em torno de uma única interface.
  - Na maioria dos casos, um testbench precisará lidar com transações em múltiplas interfaces, e isso é alcançado através do uso da virtual sequence.

Sequences UVM

- uvm_sequence_item
  - A classe base uvm_sequence_item fornece a funcionalidade básica para que objetos, tanto itens de sequência quanto sequences, operem no mecanismo de sequência.
  - set_id_info()

    Copia o sequence_id (ID da sequência) e o transaction_id (ID da transação) do item referenciado (geralmente a requisição req) para o item que chama o método (geralmente a resposta rsp).

    Deve ser sempre utilizado por drivers para inicializar respostas, garantindo compatibilidade futura.
  - is_item()
  - Get_root_sequence()
  - Get_sequence_path()

Sequences UVM

- uvm_sequence_item - Reporting interface
  - Sequence items e sequences usam o sequencer com o qual estão associados para reportar mensagens.
  - Se nenhum sequenciador tiver sido especificado para o item/sequência usando set_sequencer, ou indiretamente via uvm_sequence_base::start_item ou uvm_sequence_base::start, então o repórter global é usado.
  - uvm_get_report_object()
  - uvm_report, uvm_report_info, uvm_report_warning, uvm_report_error,and uvm_report_fatal.

    (string id, string message, int verbosity = UVM_NONE, string filename = "", int line = 0, string context_name = "", bit report_enabled_checked = 0)

Sequences UVM

- uvm_sequence_base
  - Verbosidade (Verbosity)

    UVM_INFO, UVM_WARNING, UVM_ERROR, UVM_FATAL.

    A ação tomada pelo simulador ao receber a mensagem (parar, contar erro, apenas avisar).

    INFO (menos grave) < WARNING < ERROR < FATAL (mais grave).

  - Verbosidade (Verbosity)

    A filtragem da mensagem (se ela será impressa ou suprimida).

    NONE (menos detalhe) < LOW < MEDIUM < HIGH < FULL < DEBUG (mais detalhe).

    UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG.

Sequences UVM

- uvm_sequence_base
  - Fornece as interfaces necessárias para criar fluxos de sequence itens(transactions) e/ou outras sequences.
  - Uma sequence é executada chamando seu método start, diretamente ou por meio da invocação de qualquer uma das macros `uvm_do_*.
  - set_randomize_enabled()
    Quando definido como 1, a sequence será aleatorizada automaticamente antes de ser executada pelas macros `uvm_do*
  - get_randomize_enabled()...

Sequences UVM

- uvm_sequence_base
- Tasks
  - Start()
  - Executa esta sequência, retornando quando a sequência for concluída.
  - pre_start(), pre_/post_body(), pre_do().
  - Use start para sequences.
  - Use start_item, finish_item para sequence_items.
  - Conding Guidines - cookbook

Sequences UVM

- uvm_sequence
  - Fornece as interfaces necessárias para criar fluxos de sequence itens(transactions) e/ou outras sequences.
  - Parâmetros da class REQ e RSP derivadas da uvm_sequence_item.
  - get_current_item()

    Retorna o item que está sendo executado pelo sequenciador. Se o sequenciador não estiver executando um item, retorna nulo.

    O sequencer está executando um item desde o momento em que get_next_item é chamado até o momento em que item_done é chamado.
  - get_response()

Sequences UVM

- As sequences(sequências) fornecem um mecanismo para especificar a geração de estímulos em nível de transação. Isso promove abstração e facilita o reuso de cenários.
- A execução inicia chamando o método uvm_sequence start(). Isso associa a sequence a um Sequencer e executa automaticamente a task body().
- Dentro de body(), a sequence pode gerar sequence_items para o Driver ou iniciar outras sequences (sub-sequences), criando fluxos hierárquicos, paralelos ou aleatórios.

Múltiplas Interfaces

- Uma Virtual Sequence é uma sequence que não gera itens diretamente, mas inicia sub-sequences em múltiplos sequencers diferentes.
- Ela centraliza o controle do teste, permitindo sincronizar o tráfego entre diferentes portas do DUT.
- Recomendado: A Virtual Sequence possui handles apontando para os sequencers alvo onde as sub-sequences rodarão.
- Legada: Uso de um "Virtual Sequencer" físico, que continha os handles. A abordagem moderna foca na sequence virtual segurando as referências, simplificando a hierarquia de componentes.

Mecanismo de Arbitragem

- Múltiplas sequences podem interagir concorrentemente com um único Driver. O Sequencer possui um árbitro para garantir que apenas um sequence_item acesse o Driver por vez.
- O usuário pode selecionar entre cinco algoritmos de arbitragem nativos (ex: FIFO, Prioridade Ponderada) ou criar um algoritmo personalizado. Sequences iniciadas com alta prioridade terão precedência na fila.

Roteamento de Respostas

- Quando o Driver retorna uma resposta, o Sequencer usa o campo `sequence_id` do item para roteá-la de volta à sequence correta (dentre as várias que podem estar rodando).
- O Driver deve usar a chamada `set_id_info` para garantir que a resposta mantenha o ID da requisição original.

Acesso Exclusivo (Lock e Grab)

- Para cenários críticos (como processamento de interrupções), uma sequence pode exigir acesso exclusivo ao Driver.

- Métodos lock() ou grab() bloqueiam temporariamente outras sequences, garantindo que o fluxo atual não seja interrompido.
  - Cookbook 227

Exemplo